
ZED2 DESDE PYTHON:

DETECCIÓN 3D DE POSTURAS HUMANAS

INTRODUCCIÓN

En el segundo proyecto de la asignatura se debe desarrollar una interfaz humano-máquina basada en visión 3d. Para ello, se deberán seguir dos puntos de interés de al menos una persona presente en la escena utilizando funcionalidad de Yolo, MediaPipe o ZED2. Por ejemplo, los pies o los puntos de muñeca. En el caso de no poder utilizar la funcionalidad de Yolo, MediaPipe o ZED2 se podrán añadir marcadores sobre los puntos de interés. Después, se deberá desarrollar un clasificador de gestos, definiendo un conjunto finito de gestos según las posiciones relativas de los puntos de interés que permita actuar sobre los objetos virtuales de la aplicación.

OBJETIVOS

El objetivo principal de esta práctica es analizar el funcionamiento de la detección 3D de posturas humanas con la funcionalidad de ZED2.

MATERIALES

Para desarrollar esta práctica, descargue los siguientes materiales a una carpeta de trabajo local (en adelante, “[working_folder_path](#)”) en el ordenador donde vaya a ejecutar el cuaderno (*.ipynb) o el fichero Python (*.py).

- PoliformaT -> Recursos/2-Software-&-cuadernos/122-ZED2 sample - 3D Pose tracking

Importante >> No olvide hacer una copia de seguridad del código implementado (por ejemplo, en su carpeta W:) al finalizar la práctica.

1.- ARRANQUE CON Jupyter lab

Si nunca has utilizado PyCharm, es mejor empezar con Jupyter lab. Ejecuta el programa Jupyter lab desde un terminal “Anaconda Prompt (miniconda 3)”, tras activar el “environment” adecuado.

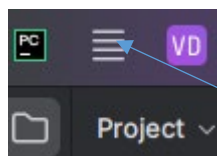
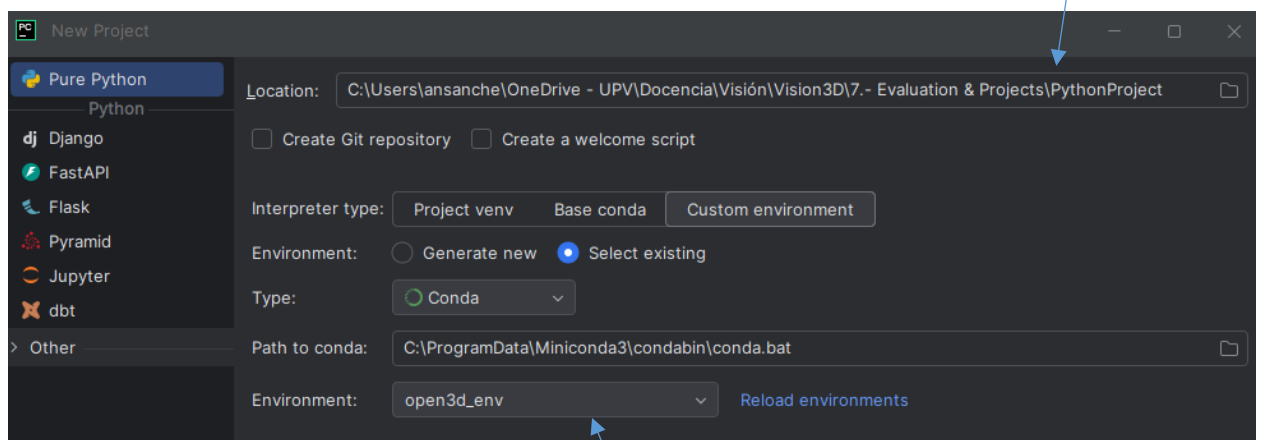
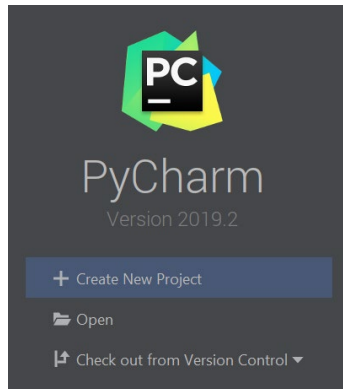
```
(base) C:\windows\system32> conda activate open3d_env
```

```
(open3d_env) C:\Users\username> jupyter lab --notebook-dir “working\_folder\_path”
```

Utilidad >> Para habilitar el autocompletado de código en JupyterLab, presione la tecla Tab mientras escribe el código.

1.2.- ARRANQUE CON PyCharm

Si prefieres comenzar a trabajar con PyCharm, deberás crear un proyecto y seleccionar el “environment” adecuado desde el desplegable “Environment”.



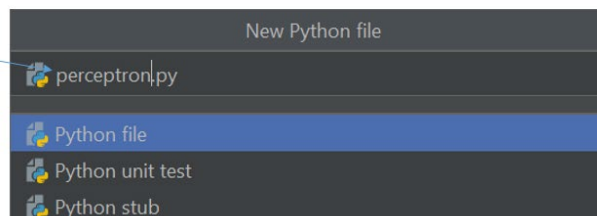
Main menu

Select environment

Para crear un nuevo fichero Python (*.py) simplemente selecciona la siguiente opción del “Menu principal”:

- Main menú -> File -> New -> Python File

New file name



DEMO 1: DETECCIÓN DE POSTURAS HUMANAS

```
"""
    This sample shows how to detect a human bodies and draw their
    modelised skeleton in an OpenGL window
"""
import cv2
import sys
import pyzed.sl as sl
import ogl_viewer_v3.viewer as gl
import cv_viewer_v3.tracking_viewer as cv_viewer
import numpy as np
import argparse

def parse_args(init): ...

def main():
    print("Running Body Tracking sample ... Press 'q' to quit, or 'm' to pause or restart")

    # Create a Camera object
    zed = sl.Camera()

    # Create a InitParameters object and set configuration parameters
    init_params = sl.InitParameters()
    init_params.camera_resolution = sl.RESOLUTION.HD1080 # Use HD1080 video mode
    init_params.coordinate_units = sl.UNIT.METER # Set coordinate units
    init_params.depth_mode = sl.DEPTH_MODE.ULTRA
    init_params.coordinate_system = sl.COORDINATE_SYSTEM.RIGHT_HANDED_Y_UP

    parse_args(init_params)

    # Open the camera
    err = zed.open(init_params)
    if err != sl.ERROR_CODE.SUCCESS:
        exit(1)

    # Enable Positional tracking (mandatory for object detection)
    positional_tracking_parameters = sl.PositionalTrackingParameters()
    # If the camera is static, uncomment the following line to have better performances
    # positional_tracking_parameters.set_as_static = True
    zed.enable_positional_tracking(positional_tracking_parameters)

    body_param = sl.BodyTrackingParameters()
    body_param.enable_tracking = True # Track people across images flow
    body_param.enable_body_fitting = False # Smooth skeleton move
    body_param.detection_model = sl.BODY_TRACKING_MODEL.HUMAN_BODY_FAST
    body_param.body_format = sl.BODY_FORMAT.BODY_18 # Choose the BODY_FORMAT you wish to use

    # Enable Object Detection module
    zed.enable_body_tracking(body_param)

    body_runtime_param = sl.BodyTrackingRuntimeParameters()
    body_runtime_param.detection_confidence_threshold = 40

    # Get ZED camera information
    camera_info = zed.get_camera_information()
    # 2D viewer utilities
    display_resolution = sl.Resolution(min(camera_info.camera_configuration.resolution.width, 1280),
    min(camera_info.camera_configuration.resolution.height, 720))
    image_scale = [display_resolution.width / camera_info.camera_configuration.resolution.width
    , display_resolution.height / camera_info.camera_configuration.resolution.height]

    # Create OpenGL viewer
    viewer = gl.GLViewer()
    viewer.init(camera_info.camera_configuration.calibration_parameters.left_cam,
    body_param.enable_tracking, body_param.body_format)
    # Create ZED objects filled in the main loop
    bodies = sl.Bodies()
    image = sl.Mat()
    key_wait = 10
```

```

while viewer.is_available():
    # Grab an image
    if zed.grab() == sl.ERROR_CODE.SUCCESS:
        # Retrieve left image
        zed.retrieve_image(image, sl.VIEW.LEFT, sl.MEM.CPU, display_resolution)
        # Retrieve bodies
        zed.retrieve_bodies(bodies, body_runtime_param)
        # Update GL view
        viewer.update_view(image, bodies)
        # Update OCV view
        image_left_ocv = image.get_data()
        cv_viewer.render_2D(image_left_ocv, image_scale, bodies.body_list, body_param.enable_tracking,
body_param.body_format)
        cv2.imshow("ZED | 2D View", image_left_ocv)
        key = cv2.waitKey(key_wait)
        if key == 113: # for 'q' key
            print("Exiting...")
            break
        if key == 109: # for 'm' key
            if (key_wait>0):
                print("Pause")
                key_wait = 0
            else :
                print("Restart")
                key_wait = 10

viewer.exit()
image.free(sl.MEM.CPU)
zed.disable_body_tracking()
zed.disable_positional_tracking()
zed.close()
cv2.destroyAllWindows()

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--input_svo_file', type=str, help='Path to an .svo file, if you want to replay
it', default = '')
    parser.add_argument('--ip_address', type=str, help='IP Address, in format a.b.c.d:port or a.b.c.d, if you
have a streaming setup', default = '')
    parser.add_argument('--resolution', type=str, help='Resolution, can be either HD2K, HD1200, HD1080,
HD720, SVGA or VGA', default = '')
    opt = parser.parse_args()
    if len(opt.input_svo_file)>0 and len(opt.ip_address)>0:
        print("Specify only input_svo_file or ip_address, or none to use wired camera, not both. Exit
program")
        exit()
    main()

```

2.- INFORMACIÓN DE LA API DE STEREO LABS

[HTTPS://WWW.STEREO LABS.COM/DOCS/BODY-TRACKING/USING-BODY-TRACKING](https://www.stereolabs.com/docs/body-tracking/using-body-tracking)

3.-TAREA

A partir del ejemplo anterior deberéis desarrollar un reconocedor de gestos e integrarlo en vuestro proyecto.